

Travail Pratique N° 1 : Initiation à l'Arduino & aux Capteurs/Actionneurs

- Le travail à rendre consiste en un rapport de TP avec captures d'écran & inclusion du code source.
- Répondre à toutes les questions et les inclure dans le compte rendu.
- Le code est contenu dans des encadrés gris
- Respecter les montages pour ne pas endommager les composants (orientation des polarités des LEDs, valeurs de résistances pour éviter la surchauffe) + ranger les composants dans leurs sachets antistatiques.
- Tout copier/coller entre groupes ou depuis des ressources bibliographiques non référencées (*ChatGPT, Gemini, etc* ...) sera vérifié visuellement et automatiquement par les outils du Professeur (*GPT output detector, Turnitin, Winston AI*) et entraînera un conseil de discipline.

Nota BENE : ce TP emploie un style similaire aux manuels d'informatique modernes (e.g., collection « Head First ») dans le but de favoriser l'apprentissage pédagogique : illustrations amusantes, messages d'encouragement, apprentissage par problèmes.

Outils professionnels : Pour ce TP, vous devez utiliser les outils suivants pour rendre votre travail :

- **Git/GitHub** : Versionner les fichiers Arduino, schémas et rapports. Utiliser quelques branches (main, dev, feature/...). Faire quelques "push" avec des commits clairs. Utiliser des Pull Requests (PR) avec relecture par un membre du groupe.
- **Jira** : Créer un tableau/projet avec quelques cartes/tickets pour les tâches (montage, code, test...). Chaque ticket doit contenir une description claire et être mis à jour. Lier quelques commits Git aux tickets.
- **Slack** : Documenter quelques échanges techniques (questions, problèmes, décisions) au sein du groupe.
- **CI/CD** : Mettre en place un pipeline simple vérifiant au minimum la compilation du code Arduino pour quelques commit.
- **Tests** : Documenter quelques scénarios de test (entrées/sorties attendues, cas d'erreur). Fournir des preuves (logs série, captures).

Votre rapport final devra inclure les captures d'écran de tous les outils précédents. Tout TP sans preuve d'utilisation de ces outils sera considéré incomplet et pénalisé.

Objectif du TP



Le TP a pour but d'initier à la programmation de la carte Arduino. Il s'agira plus précisément de (a) s'appropriier le kit Arduino fourni, (b) apprendre à lire/afficher des données (port série), et de (c) capter/agir sur l'environnement extérieur.

Question préliminaire (inventaire) :

À partir du matériel fourni, dresser une analyse des composants sous forme de tableau en indiquant leur rôle, type de signal, interface et principales contraintes. Ensuite, dans cet inventaire, expliquer le fonctionnement d'un composant que vous préférez.

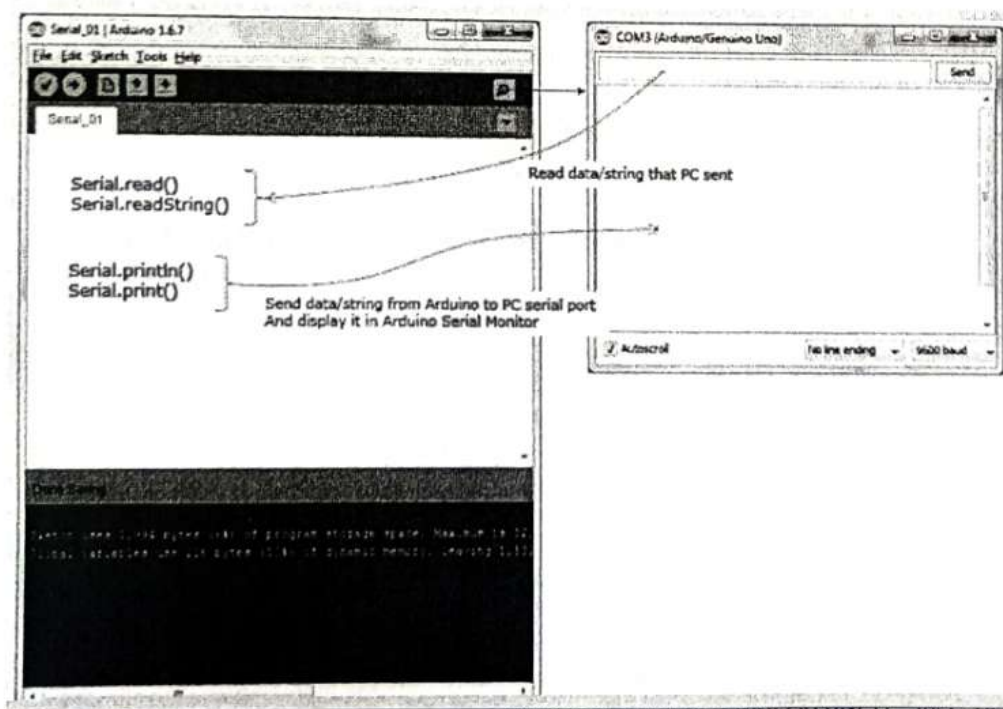
Configurer maintenant le PC pour programmer l'Arduino (installation de l'IDE, installation drivers, etc ...). Expliquer les étapes suivies avec captures d'écran du « build réussi ». Capturer et expliquer la sortie console d'un « sketch » compilé et téléversé avec succès.

Expliquer le rôle du « bootloader » dans une carte Arduino et ce qu'il se passe concrètement au démarrage entre l'alimentation de la carte et l'exécution du programme utilisateur.

Exercice 1 : PORT SÉRIE (I/O avec l'Arduino)



L'objectif de cet exercice est d'apprendre à utiliser le port série avec l'Arduino. Nous allons voir comment envoyer puis recevoir des informations avec l'ordinateur.



Materiel:

- 1 * Arduino
- 1 * cable USB A => USB B
- 1 * PC

Code source

```
/* -----  
Impression de caracteres par l'Arduino  
----- */  
  
void setup(){  
  Serial.begin(9600);  
}  
  
void loop() {  
  Serial.println("Salam alaykoum");  
  Serial.println("Azul Fellawn");  
  Serial.println("Bonjour à tous");  
  delay(1000);  
}
```

1. Expliquer le rôle de la couche de communication entre USB et UART dans une carte Arduino et ce qu'elle permet concrètement de faire.
2. Comment le choix du débit de communication (baud rate) influence-t-il la fiabilité des échanges entre un PC et une carte Arduino ?
3. Prédire le fonctionnement du code source ci-dessous. Son exécution confirme-t-elle votre prédiction ?

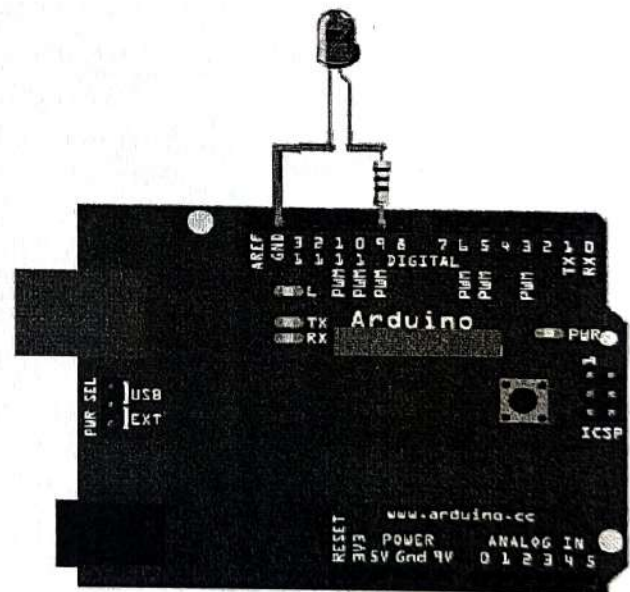
Code source

Materiel:

- 1 * Arduino
- 1 * led
- 1 * resistance
- 1 * cable USB A => USB B
- 1 * PC

```
/*-----  
Envoi et reception d un message du port serie  
-----*/  
  
int led = 9;  
char octet_rx = 0;  
  
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  if (Serial.available() > 0) {  
    octet_rx = Serial.read();  
    delay(5000);  
    if (octet_rx == 'a') /* a comme allumer */  
    { digitalWrite(led, HIGH);  
      Serial.println("LED allumee");  
    }  
    if (octet_rx == 'e') /* e comme eteindre */  
    {  
      digitalWrite(led, LOW);  
      Serial.println("LED eteinte");  
    }  
  }  
}
```

1. À partir d'une LED du kit, identifier les paramètres électriques importants à prendre en compte pour son utilisation sécurisée dans un système embarqué.
2. Réaliser sur le « breadboard » un montage qui reproduit la configuration de la figure ci-dessous. Calculer la résistance nécessaire à partir de la tension V_{cc} , de la V_f de la LED, et du courant choisi (e.g., 10–20 mA).
3. Exécuter le code ci-dessus et l'expliquer. Allumer et éteindre la LED en entrant les caractères appropriés sur le port série.
4. Concevoir une extension du programme permettant de piloter la LED via plusieurs commandes textuelles (allumer, éteindre, clignoter, inverser), tout en garantissant la robustesse de la lecture série (gestion des caractères parasites et retours chariot).
5. Dans une communication série entre un PC et une carte Arduino, expliciter le sens de circulation des données lors des opérations de lecture et d'écriture, et le rôle respectif de chaque extrémité.
6. Concevoir un système de contrôle de deux LEDs dont les rythmes d'activation sont paramétrables dynamiquement via le port série (périodes P1 et P2 en ms), tout en garantissant une exécution non bloquante et une gestion robuste des entrées utilisateur.



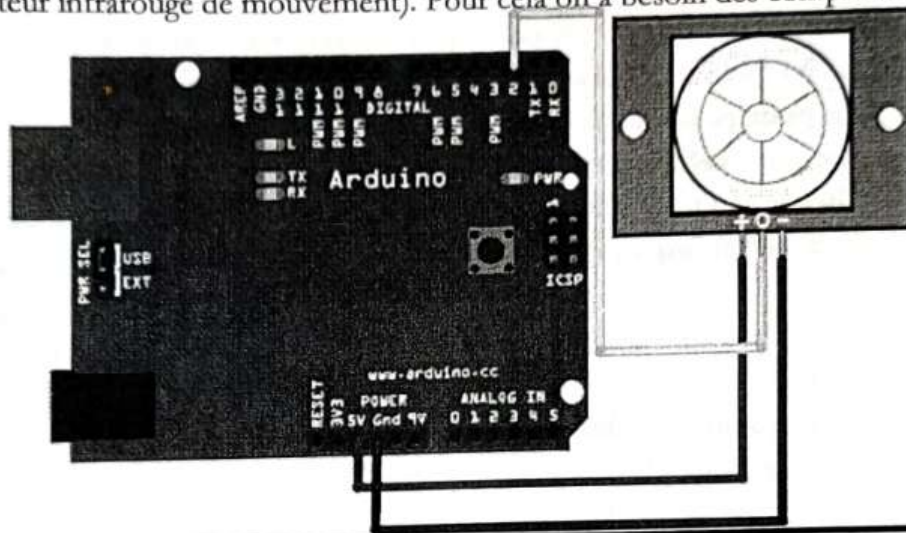
Exercice 2 (Capteurs) : Pour le meilleur et pour le PIR ...



Le but de cet exercice est de savoir comment utiliser le capteur PIR (Passive Infrared Sensor : détecteur infrarouge de mouvement). Pour cela on a besoin des composants suivants :

Matériel:

- 1 * Arduino
- 1 * Capteur PIR
- 3 * Câbles Mâle /Mâle



Code source:

```
int pinEntree=2;           // choisir la broche d entree (pour capteur PIR)
int etatPir=LOW;           // on suppose aucun mouvement detecte
int val=0;                 //variable pour la lecture du statut de la broche

void setup()
{
  pinMode(pinEntree,INPUT); //declarer le capteur comme entree
  Serial.begin(9600);
}

void loop()
{
  val=digitalRead(pinEntree); // lire la valeur entree
  if(val==HIGH){              // verifier si l entree est haute
    if (etatPir==LOW){        // on vient d allumer
      Serial.println("Mouvement detecte!"); //imprimer a chaque changement de sortie
      etatPir==HIGH;
    }
  }else{
    if (etatPir==HIGH){       //le mouvement vient de s arreter
      Serial.println("Mouvement termine!"); //imprimer a chaque changement
      etatPir==LOW;
    }
  }
}
```

1. Rechercher le *datasheet* du capteur PIR utilisé. Identifier et commenter les éléments clés – principe de détection, brochage, réglages et modes de fonctionnement.
2. Réaliser le montage sur la figure et écrire le programme. Activer également le « repeatable trigger » sur le PIR et analyser son comportement dans le temps. Produire (1) un chronogramme complet ; (2) des captures du moniteur série sur plusieurs activations successives ; (3) une analyse comparative entre « single trigger » et « repeatable trigger »
Proposer un code source qui permet de visualiser sur la LED que la durée d'allumage est différente en fonction du mode « repeatable trigger » (ou pas).
3. Étudier le principe physique du capteur PIR. Expliquer comment la variation de rayonnement infrarouge est transformée en signal électrique exploitable. Faire le lien avec une architecture de type comparateur différentiel à deux zones de détection IR et expliquer pourquoi cela permet de détecter un mouvement plutôt qu'une température absolue.
4. Que se passe-t-il si on enlève les conditions « `etatPir = HIGH` » et « `etatPir = LOW` » ? Quel est leur rôle ?
Proposer une solution robuste intégrant :
 - un «debounce» logiciel
 - une time window de validation d'événement
 - une justification claire de leur intérêt en système embarqué réel



5. **Système d'alarme :** On veut construire un système d'alarme en ajoutant en plus un buzzer actif. Lorsque le capteur PIR détecte un mouvement, la LED va s'allumer et le buzzer va déclencher une sonnerie.
 - Réaliser le montage de ce circuit (schéma à l'aide d'un outil tel que Fritzing, Tinkercad ou autre) et écrire le programme.
 - Implémenter un anti-spam pour le « buzzer » : sonner N secondes puis s'arrêter.
 - Implémenter un contrôle pour armer et désarmer le buzzer via le port série → « A » = armer (la LED & Buzzer réagissent au PIR), « D » = désarmer (ne rien faire même si le PIR détecte)
6. **Question pour aller plus loin :** Connecter le maximum de capteurs que vous pouvez, depuis le kit que je vous ai fourni.

Pour chaque capteur que vous choisissiez :

- Extraire de la *datasheet* les informations critiques : principe physique, plage de mesure, résolution, précision, limites d'usage.
- Implémenter un code de test issu d'une source technique fiable et l'adapter si nécessaire. Expliquer brièvement le fonctionnement du code.
- Identifier les grandeurs physiques mesurées.
- Déterminer le mode de communication utilisé (I2C, SPI, UART, analogique/numérique simple) et justifier techniquement ce choix (avantage / contrainte).

Bravo, vous êtes maintenant experts en capteurs Arduino ! 😊

Exercice 3 (Actionneurs) : Moteurs

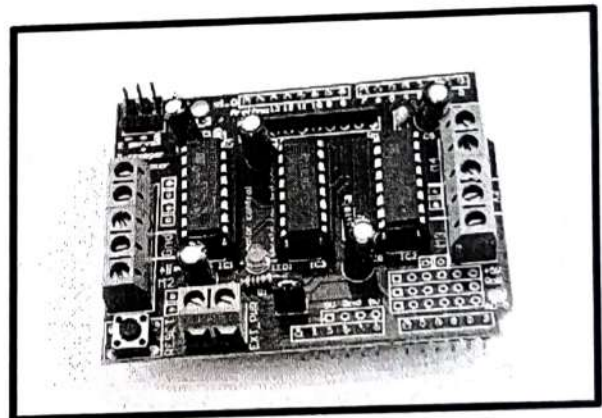


Le but de cet exercice est de contrôler la vitesse de moteurs par « input » utilisateur.

Pour cela on a besoin des composants suivants :

Materiel:

- 1 * Arduino UNO
- 1 * Moteur DC 6V
- 1 * hélice
- 1 * shield Moteur
- 1 * Plaque d'essai
- * Câbles Mâle /Mâle



Exemple de Shield Moteur

Code source:

```
#include <AFMotor.h>

AF_DCMotor motor(4);

void setup()
{
    //Vitesse initiale
    motor.setSpeed(200);
    motor.run(RELEASE);
}

void loop()
{
    uint8_t i;

    // Allumer
    motor.run(FORWARD);

    // Accelerer
    for (i=0; i<255; i++)
    {
        motor.setSpeed(i);
        delay(10);
    }

    // Decelerer
    for (i=255; i!=0; i--)
    {
        motor.setSpeed(i);
        delay(10);
    }

    // changer de direction
    motor.run(BACKWARD);

    // Accelerer
    for (i=0; i<255; i++)
    {
        motor.setSpeed(i);
        delay(10);
    }

    // Decelerer
    for (i=255; i!=0; i--)
    {
        motor.setSpeed(i);
        delay(10);
    }

    // Eteindre
    motor.run(RELEASE);
    delay(1000);
}
```


1. Analyser le shield (en lisant son « datasheet ») : nom exact, composants principaux, courant max par canal, pont en H utilisé, protection thermique, alimentation requise. Etudier la librairie associée ? Quelles sont les limites de ce shield dans le cas d'un usage industriel (e.g., robotique) ?



2. **Ventilateur** : en utilisant le shield moteur mis à votre disposition, connecter le moteur DC à l'Arduino et construire un mini ventilateur (utiliser aussi l'hélice 3 pales). Le contrôle par PWM via port série : l'utilisateur peut entrer une valeur 0-255 pour ajuster la vitesse.
3. Structurer le programme sous forme de machine d'état pour gérer proprement 4 modes OFF/ECO/NORMAL/TURBO via le port série.
4. Implémenter une rampe de vitesse progressive au lieu d'un changement brutal du PWM, afin de lisser la montée et la descente de vitesse.
5. Ajouter un mécanisme de sécurité logicielle (détection d'entrées aberrantes) pour rendre le système plus robuste.
6. Concevoir une petite application embarquée permettant de piloter à la fois un servo moteur et un moteur pas à pas, via une interface série unifiée ?
 - o Pour le servo : contrôler la position absolue, avec au moins 2 positions accessibles par le port série.
 - o Pour le moteur pas à pas : contrôler vitesse et sens de rotation à partir des commandes utilisateur.

Quel est le lien avec ce qu'on a vu en cours ?

2. **Question pour aller plus loin** : Concevoir un système embarqué où la valeur d'un capteur (au choix) permet de piloter de manière stable et précise la vitesse, la position et/ou la direction des moteurs. Essayez d'être créatifs.



Bravo, vous êtes maintenant un expert en actionneurs Arduino !

■ Fin du TP